1 Brief description

A stochastic cellular automata running a 1k-RAM, 30kHz 6502 CPU pseudo-asynchronously at each cell.

2 Introduction

A cellular automata where each cell runs a virtual 6502, a classic CPU released in 1975. The appeal to retro computing is designed to win a wide audience, to support which we will also develop mobile/serverless app infrastructure to allow mobile code to run on peoples' phones and hop between phones, with mutation and selection. A client viewer layer will allow visual interpretation of virtual memory, as well as local policies e.g. migration to/from server (and/or local neighborhood) or suppression of particular programs. The long-term goal is to obtain a dataset that will allow us to ask whether we can use the tools of population genetics and phylogenetics to study mobile code on a large scale. Subprojects will include the design of various agent-based and cellular automata models from physics, biology, chemistry, and other fields of science and A-Life. To motivate broad adoption we will offer prizes for the most creative and most virally successful programs.

- The paged storage is addressed as a square array of cells with $B = 256$ cells per side and $M = 1024$ bytes per cell, so there are $B^2 = 65,536$ (64k) cells requiring $B^2M = 64$Mb of storage.

- The memory-mapped neighborhood is 7x7 cells around the local origin (0,0). Cell n occupies the space from $1024n$ bytes to $1024n + 1023$. The relationship between cell index, memory location, and offset (x, y) are shown in these tables

  Memory pages.

	$x=-3$	$x=-2$	$x=-1$	$x=0$	$x=1$	$x=2$	$x=3$
$y=3$	C0..C3	B0..B3	90..93	64..67	74..77	94..97	B4..B7
$y=2$	AC..AF	60..63	50..53	24..27	34..37	54..57	98..9B
$y=1$	8C..8F	4C..4F	20..23	04..07	14..17	38..3B	78..7B
$y=0$	70..73	30..33	10..13	00..03	08..0B	28..2B	68..6B
$y=-1$	88..8B	48..4B	1C..1F	0C..0F	18..1B	3C..3F	7C..7F
$y=-2$	A8..AB	5C..5F	44..47	2C..2F	40..43	58..5B	9C..9F
$y=-3$	BC..BF	A4..A7	84..87	6C..6F	80..83	A0..A3	B8..BB

  Each time execution resumes after an interrupt, the origin is resampled and the neighborhood rotated by a random multiple of 90 degrees. Zero page locations 0xF0-0xF9 are designated as "oriented registers", whose top 6 bits are rotated along with the memory map when a different orientation is sampled. (Note: 0xF9 is the storage location for PCHI, so this is also rotated.)

  Cell numbers:

	$x=-3$	$x=-2$	$x=-1$	$x=0$	$x=1$	$x=2$	$x=3$
$y=3$	48	44	36	25	29	37	45
$y=2$	43	24	20	9	13	21	38
$y=1$	35	19	8	1	5	14	30
$y=0$	28	12	4	0	2	10	26
$y=-1$	34	18	7	3	6	15	31
$y=-2$	42	23	17	11	16	22	39
$y=-3$	47	41	33	27	32	40	46

Cell coordinates:

Range	Directions
0	Origin
1..4	N=(0,+1), E=(+1,0), S, W
5..8	NE, SE, SW, NW
9..12	N^2, E^2, S^2, W^2
13..20	N^2E, NE^2, SE^2, S^2E, S^2W, SW^2, NW^2, N^2W
21..24	N^3, E^3, S^3, W^3
25..27	N^2E^2, S^2E^2, S^2W^2, N^2W^2
28..35	N^3E, NE^3, SE^3, S^3E, S^3W, SW^3, NW^3, N^3W
36..43	N^3E^2, N^2E^3, S^2E^3, S^3E^2, S^3W^2, S^2W^3, N^2W^3, N^3W^2
44..48	N^3E^3, S^3E^3, S^3W^3, N^3W^3

- The area from 0xE000-0xEE3F contains some read-only lookup tables for mapping various transformations inside the neighborhood (operations that yield vectors outside the neighborhood return 0xFF)

Table start S	Meaning of $S[i]$
0xE000 $+64j$	Vector addition $v_i + v_j$
0xEC40	Rotation 90° clockwise
0xEC80	Rotation 180°
0xECC0	Rotation 270°
0xED00	Reflection about x-axis
0xED40	Reflection about y-axis
0xED80	X coordinate of v_i plus 3
0xEDC0	Y coordinate of v_i plus 3
0xEE00	See below

  The table from 0xEE00-0xEE3F contains the mapping from (x, y) coordinates to cell indices, with y ascending fastest, starting from cell (-3,-3); so the byte in 0xEE00+$(y + 3) + 64(x + 3)$ is the index of cell with relative offset (x, y) for $-3 \leq x, y \leq 3$.

- We aim to clock the entire board at the rate of a 1981 BBC Micro (2Mhz), so each cell updates at 30.5Hz. (This is a lower bound. There is no reason you can't run the board faster.)

- Interrupts arrive as an (approximately) Poisson process with an average rate of 1 per 4,096 cycles. An interrupt is handled as follows:

  - The CPU registers are written to the last seven bytes of zero page.
  - If the interrupt disable flag (I) in the status register P is clear, then the memory-mapped neighborhood of the current origin is copied from RAM to storage (after un-rotating the oriented registers from 0xF0-F9). Otherwise, if I is set, all edits made since the last interrupt are lost. (Note that this step requires that either the operating system or the paging hardware must "remember" the current origin and orientation between interrupts, along with the pre-edited state of the memory-mapped neighborhood. This information should not be visible to user-space code.)
  - A new origin cell (i, j) and orientation is randomly sampled. The memory-mapped neighborhood of that cell is copied from long-term storage to RAM, using the sampled orientation. Oriented registers at bytes 0xF0-0xF9 of every 1K block have rotations applied.
  - The last four bytes of zero page are overwritten with pseudorandom numbers.
  - CPU registers are restored from the last seven bytes of zero page.

- A BRK software interrupt is handled almost identically, with the following differences:
  - The B flag is set before P is pushed to the stack.
  - After saving registers Y,X,A,S to memory, the operand byte b (the byte in memory immediately following the BRK opcode) is examined. Let $N^2 = 49$ (the number of cells in the 7x7 neighborhood). The operand encodes a source cell $s = \lfloor b/N^2 \rfloor$ and a destination cell $d = b \bmod N^2$, where s ranges over cells 0–4 (the origin and its four cardinal neighbors) and d ranges over cells 0–48 (any cell in the neighborhood).

    If $1 \leq b < 245$ and $s \neq d$, cells s and d are swapped (1024-byte blocks each). Otherwise ($b = 0$, $s = d$, or $b \geq 245$) nothing happens. In all cases, control returns to the scheduler.

    Instant cell swap is justifiable as implementing diffusion. A future extension may add error-prone copy (justifiable on thermodynamic grounds and/or by Shannon's noisy channel coding theorem), with a board-specific error rate parameter.

    The guiding principle when considering adding more software interrupts, or expanding the OS in any way, should be to not add any functionality that can't be justified as "physics".
  - After any swap is performed, control returns to the scheduler, which will randomly pass control to another cell.
  - The interrupt disable flag is ignored by BRK; memory is always copied back to storage following a software interrupt.

- A software interrupt due to a bad instruction is handled like a BRK 0 (nothing happens, control returns to scheduler).

3 Links

- Avida (**?**; **?**) `https://en.wikipedia.org/wiki/Avida_(software)`
- Core War `https://en.wikipedia.org/wiki/Core_War`
- The BBC Micro Bot `https://mastodon.me.uk/@bbcmicrobot`